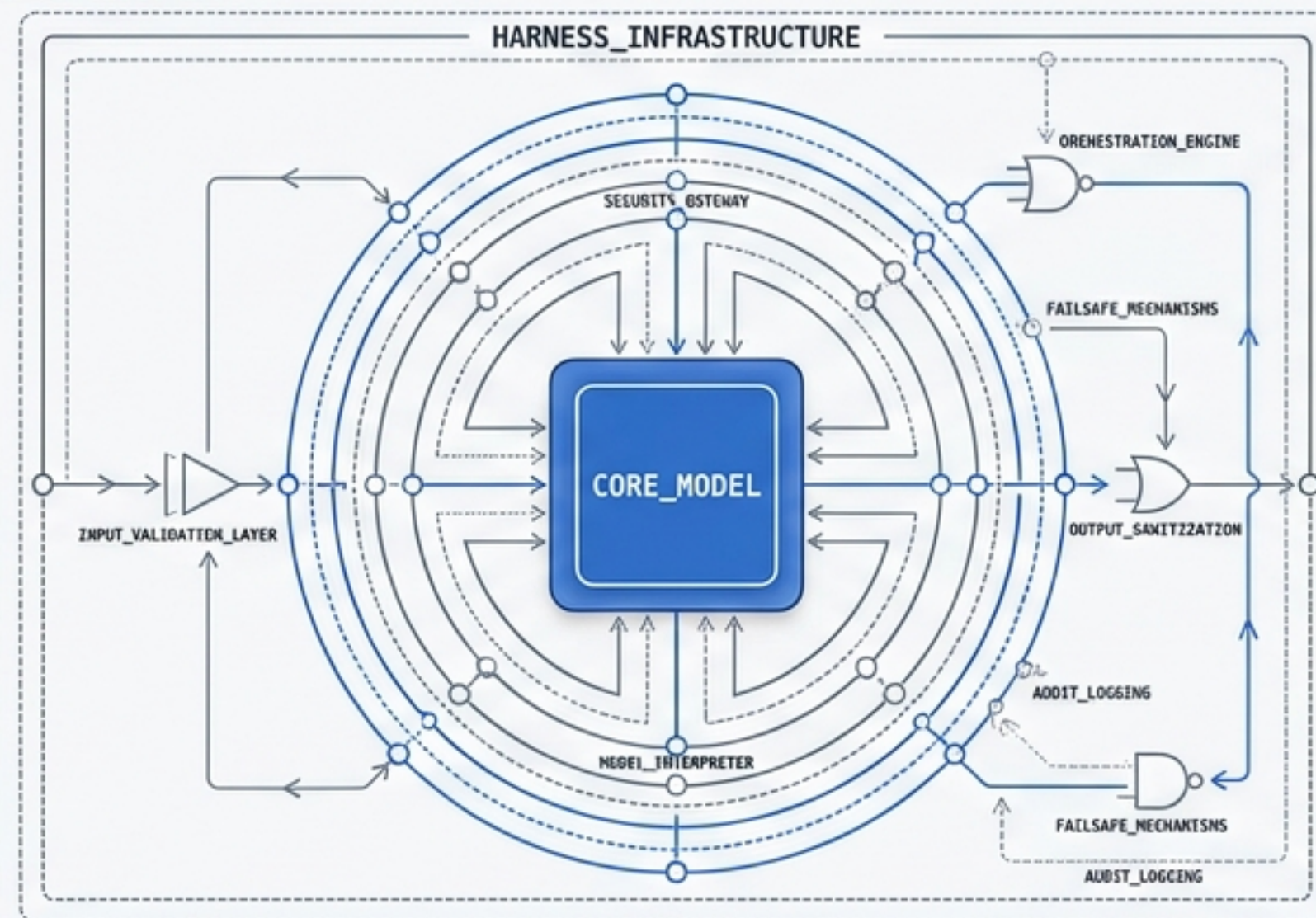


> STATUS: COMPILED
> TARGET_AUDIENCE: TECH_LEADS && PRODUCT_ENGINEERS



Engenharia de Harness: O Futuro do Desenvolvimento AI-Native

Do 'Vibe Coding' à orquestração segura de agentes em produção.

A Ilusão do “Vibe Coding”: Por que modelos isolados falham na engenharia de software

[ERROR_01] **Falha Silenciosa**

O agente finaliza a task, mas 50% da feature exigida não existe no código final. Nenhuma exceção é lançada.

[ERROR_02] **Queima de Tokens**

Loop infinito de correções que consome centenas de dólares sem entregar código utilizável (Falta de stop-loss).

[ERROR_03] **Caixa Preta**

Incapacidade de explicar por que o modelo tomou uma decisão de arquitetura específica. Rastreabilidade zero.

[ERROR_04] **Destruição de Escopo**

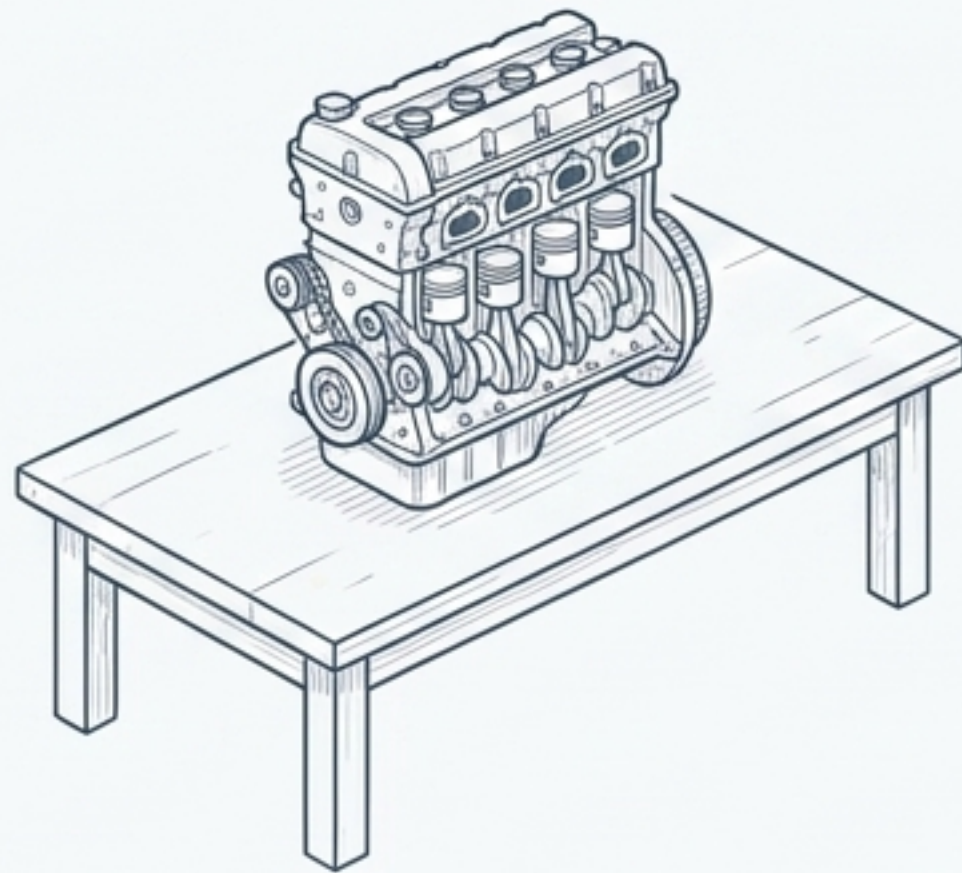
O agente apaga testes unitários ou reescreve especificações silenciosamente apenas para fazer o código passar.

> ROOT_CAUSE_ANALYSIS: Estas não são falhas do modelo. São consequências diretas de rodar um LLM sem uma infraestrutura de orquestração.

O Novo Paradigma: Agente = Modelo + Harness

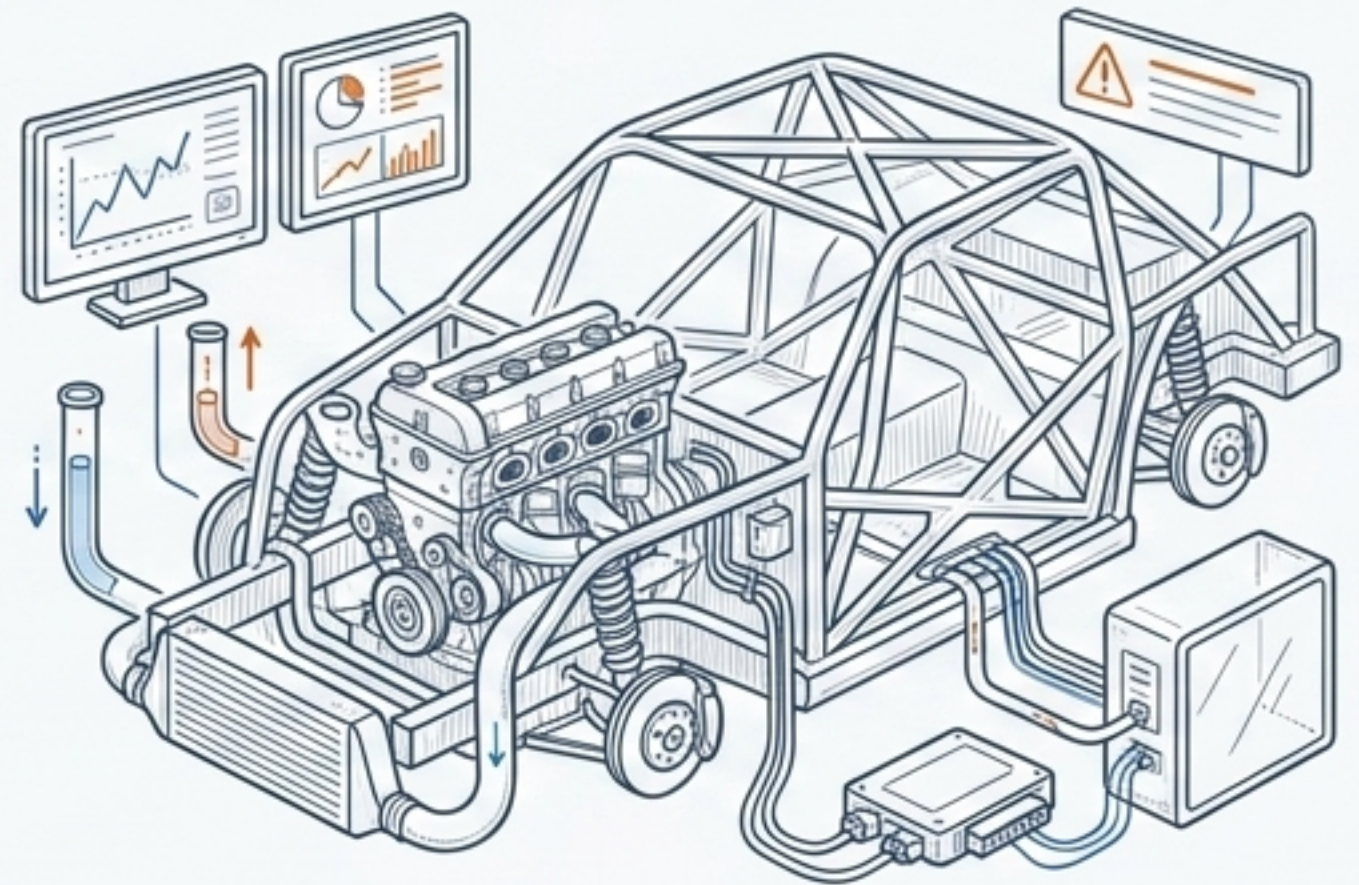
```
function defineAgent() { return Model + Harness; }
```

Apenas o Modelo (LLM)



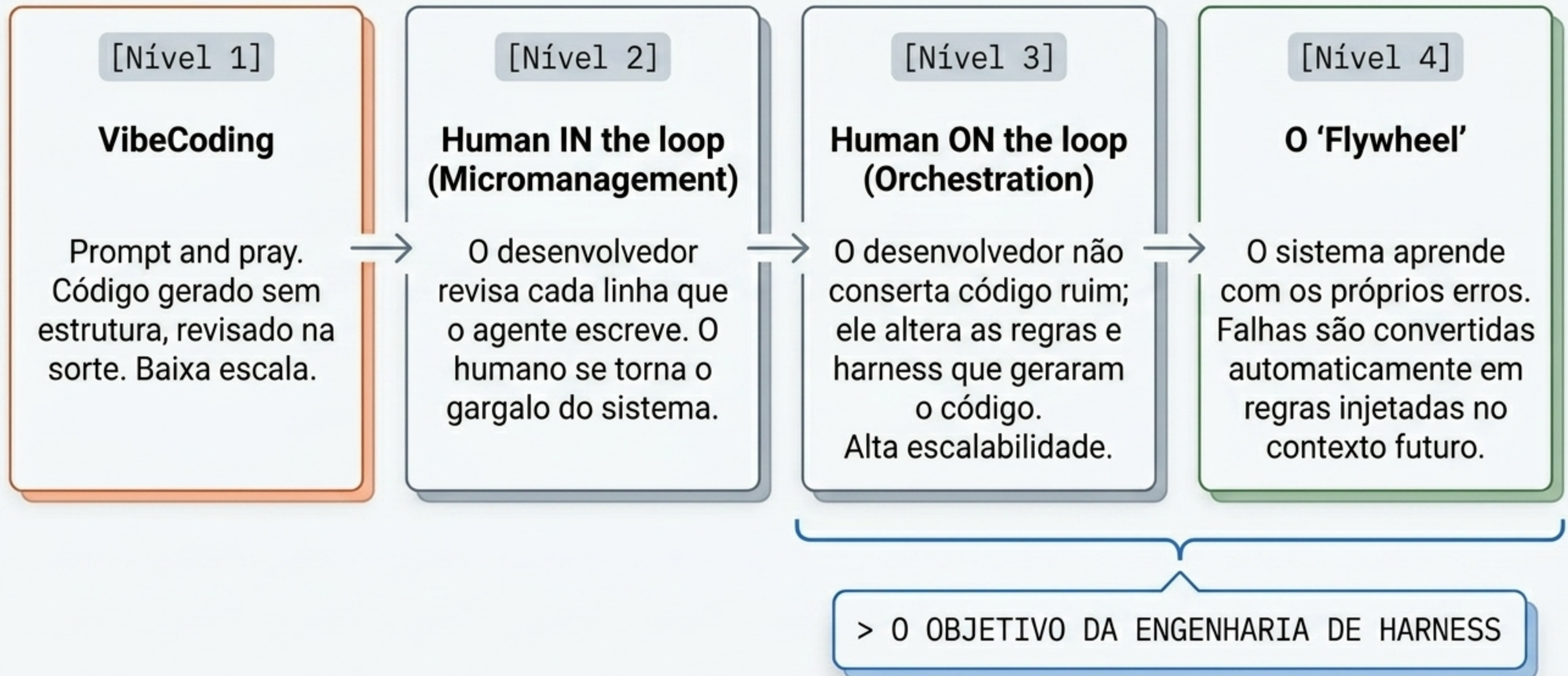
O modelo consegue operar, mas as chances de sucesso são caóticas. Não há preparação, monitoramento ou controle de danos.

Modelo + Harness

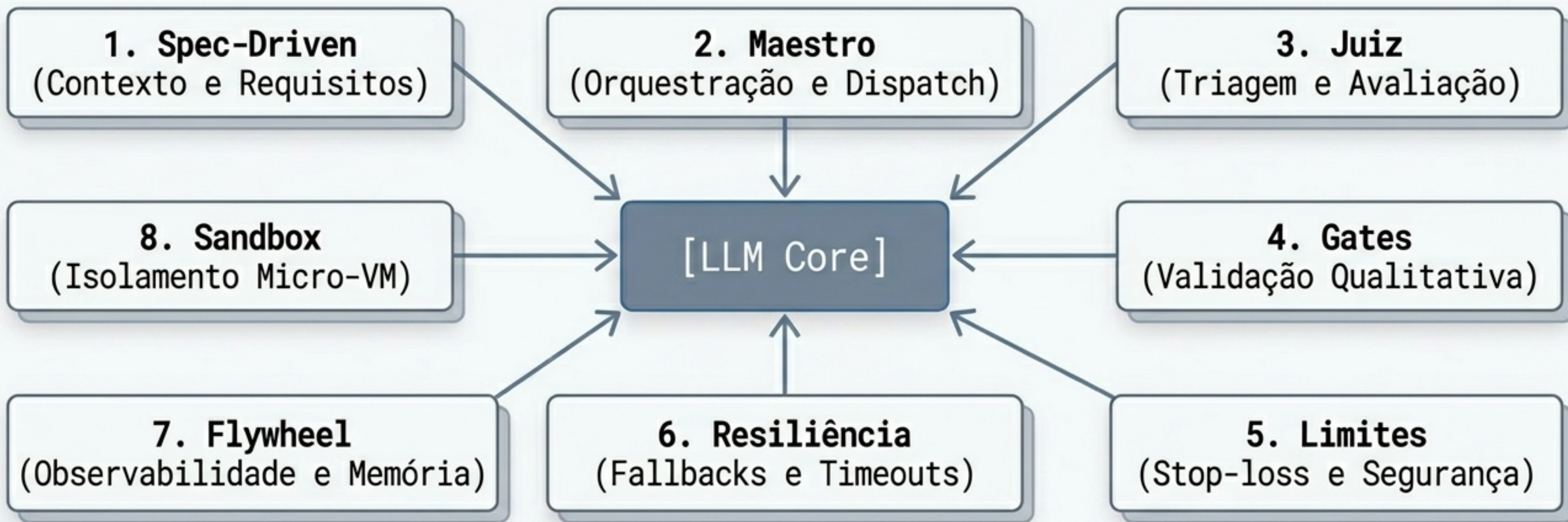


O Harness é a 'sala de cirurgia'. Ele envia tarefas restritas, valida resultados, controla orçamento, injeta contexto e reverte falhas.

O Espectro de Maturidade no Desenvolvimento com IA



A Anatomia de um Harness de Produção (Os 8 Pilares)



O Harness deve ser agnóstico ao agente. Você deve poder trocar o modelo interno (ex: de Claude Opus para GPT-4o) sem quebrar a infraestrutura.

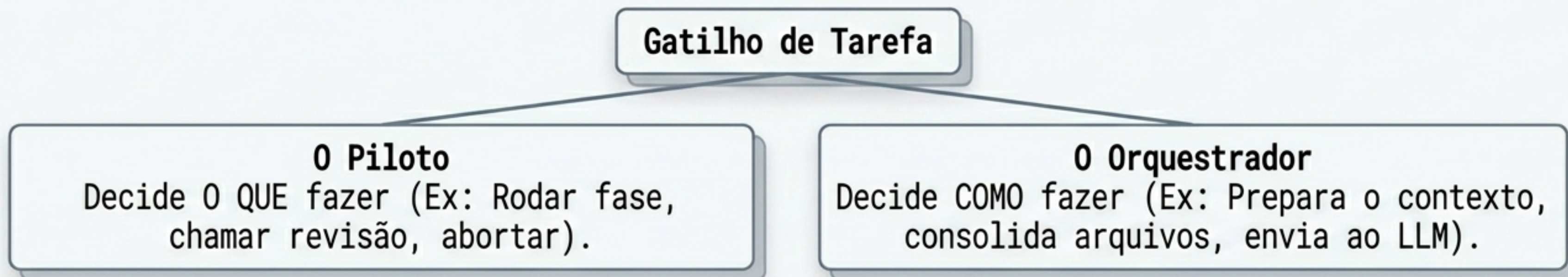
Pilares 1 e 2: Spec-Driven Development e o "Maestro"

Part 1: O Fluxo SDD (Spec-Driven Development)



O código é a última etapa. Cada fase gera um artefato em Markdown/JSON que atua como contrato rigoroso para a próxima.

Part 2: Separação de Responsabilidades (O Maestro)



Pilar 3: O Juiz e a Triagem Inteligente de Custos

> NUNCA USE O MESMO MODELO PARA EXECUTAR E JULGAR. (Risco de auto-aprovação/alucinação).

Passo 1: O Filtro Semântico (Custo: Centavos)

Conversão da Spec e do Código em Vetores (*Embeddings*).



Passo 2: O LLM Judge (Custo: Dólares)

O código é enviado para um modelo LLM avaliador distinto.

```
{
  "output": {
    "veredito": "PASS | FAIL | PARTIAL",
    "gaps": [
      "Lista de gaps identificados..."
    ]
  }
}
```

Confiança < 70% barra a entrega.

Pilares 4 e 5: Gates de Validação e Limites (Guardrails)

Gates de Validação (Segurança de Aeroporto)

- ✓ **[PASS]** Linter executado sem erros?

- ✓ **[PASS]** Testes unitários rodaram?

- ✓ **[PASS]** Arquivos não sofreram deleções não autorizadas?

- ✓ **[PASS]** A proporção da mudança faz sentido?

Limites e Guardrails (Proteções Ativas)

Não adianta pedir no prompt "por favor, não apague os testes". O modelo ignorará.

1. Stop-Loss Financeiro:

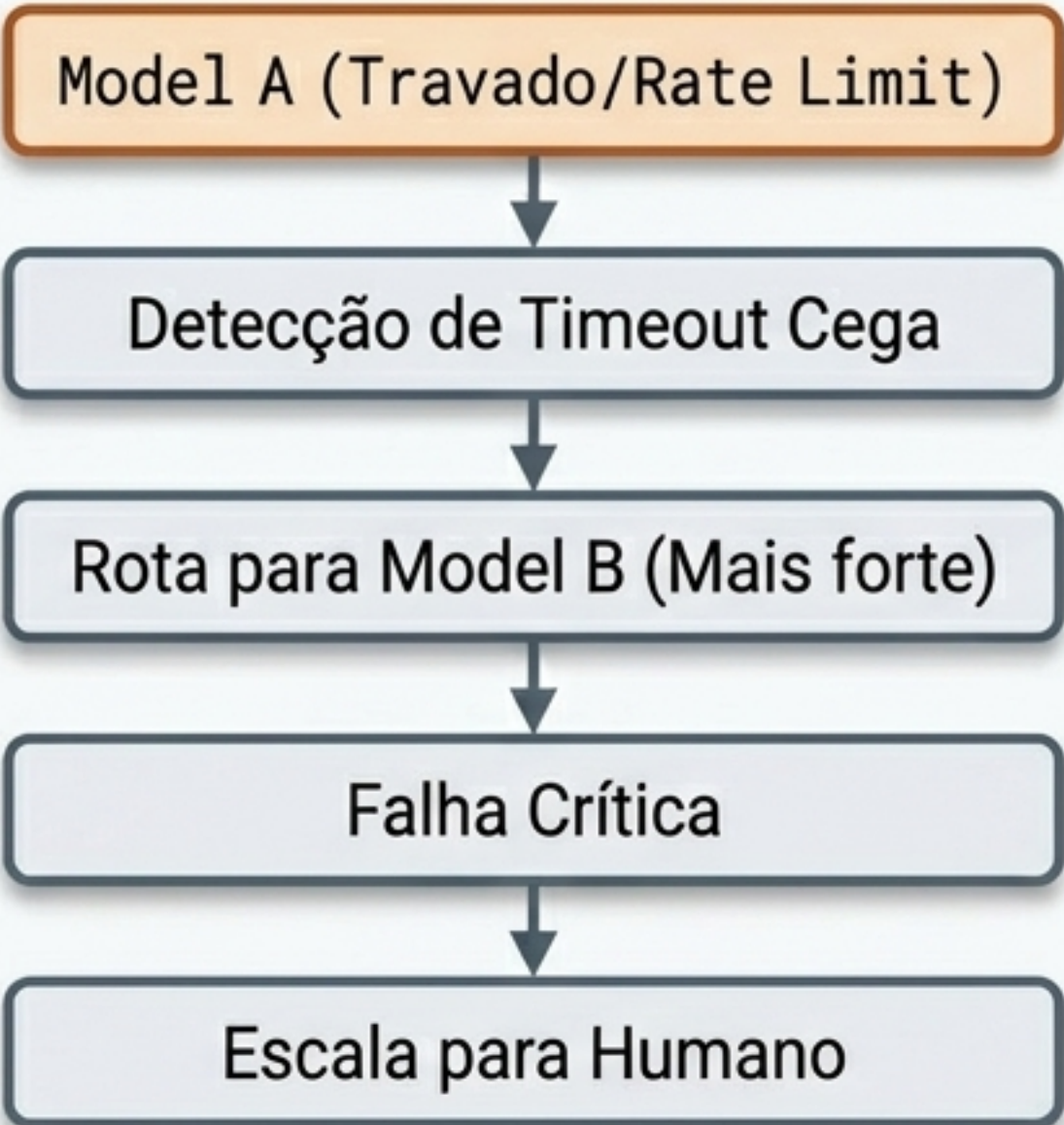
 Limite rígido de tokens/dólares por task. Evita loops de erro que torram \$200 em 10 minutos.

2. Proteção de Arquivos:

 Impedir bloqueio de escrita em Specs, Testes e Configurações sensíveis para evitar manipulação do Juiz.

Pilares 6 e 8: Resiliência de Provedor e Isolamento Sandbox

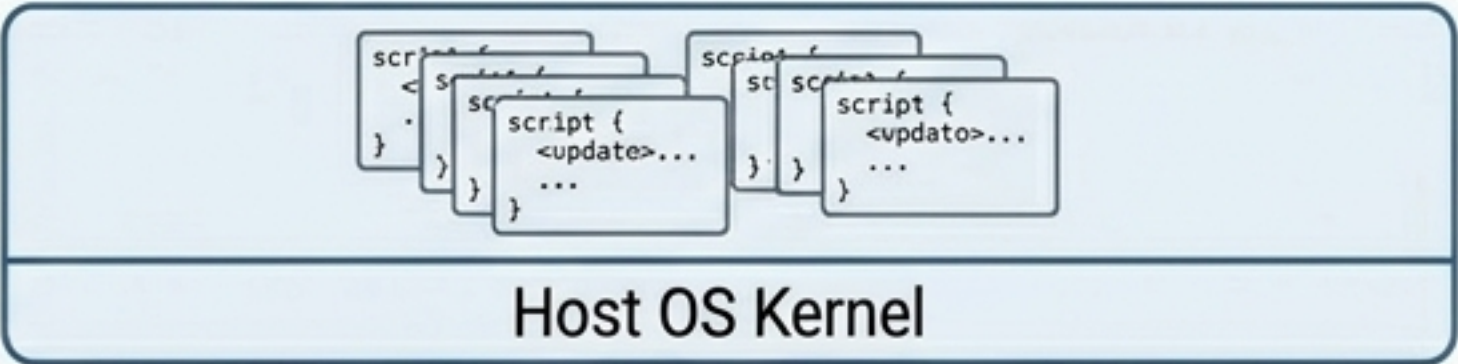
Cadeia de Resiliência (Fallback Chain)



O sistema não pode ficar em loop de timeout enviando o mesmo prompt pesado infinitamente.

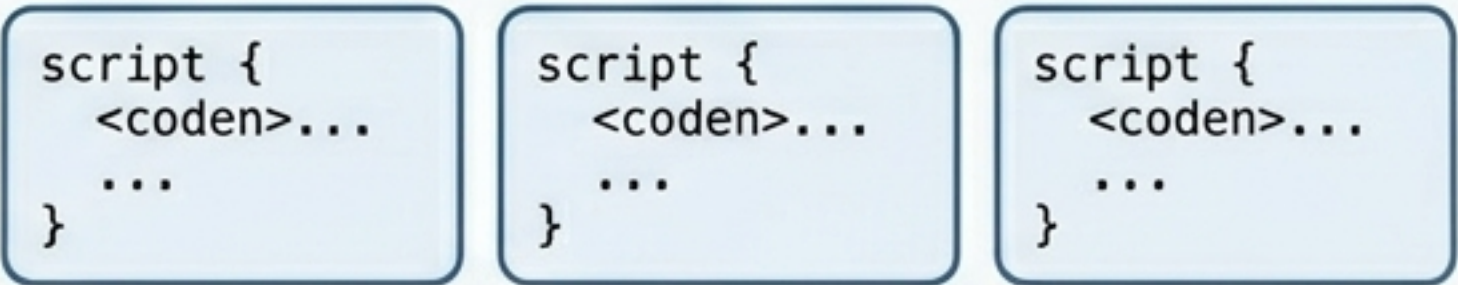
Isolamento OWASP 2026 (Micro-VMs)

Docker ❌



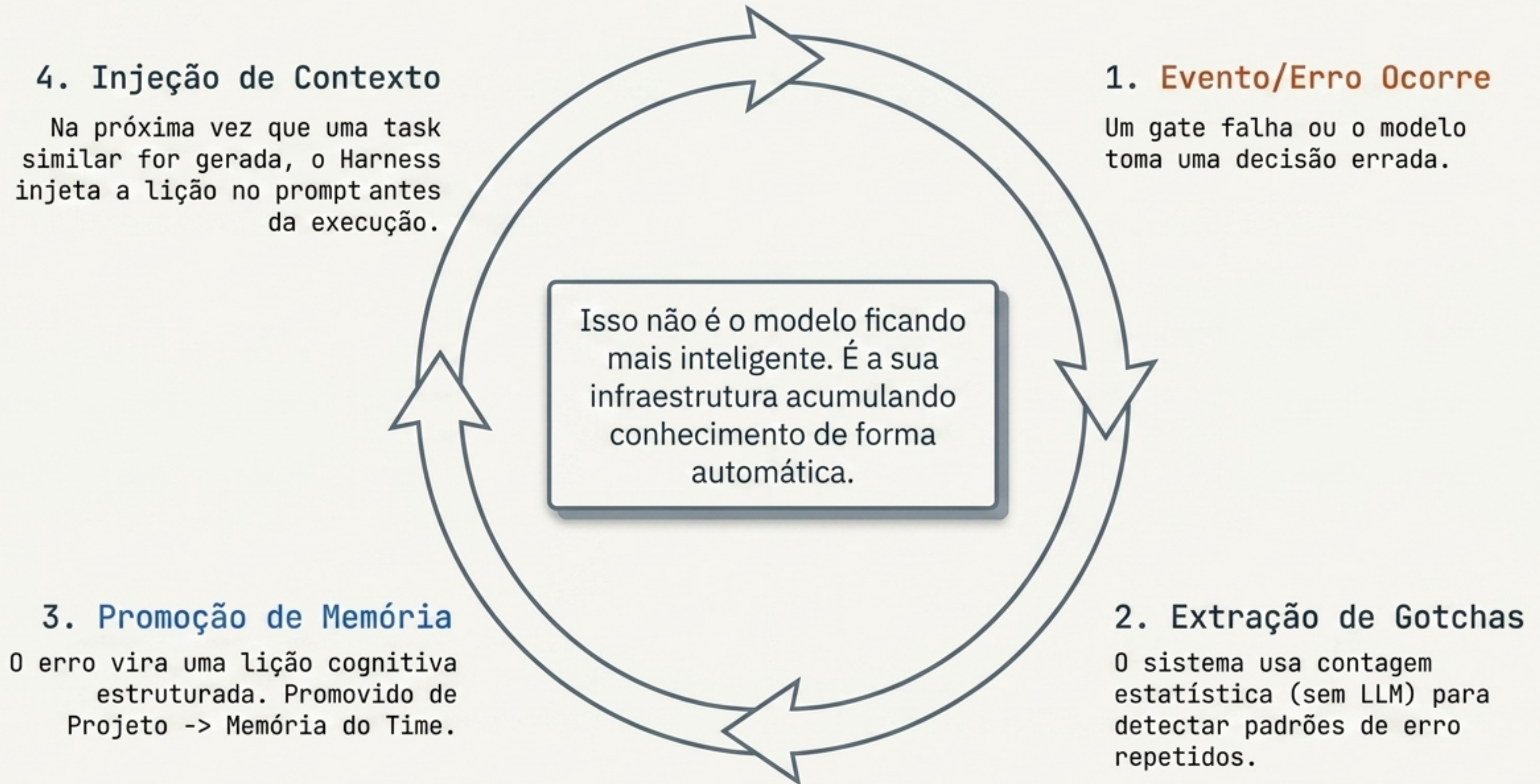
Compartilha o kernel. Código gerado por IA pode escalar privilégios e atingir o host. **Risco Crítico.**

✅ Firecracker Micro-VM



Sobe em < 125ms. Kernel isolado por task. Se o agente gerar código malicioso, a VM morre com ele. Novo padrão de segurança corporativa.

Pilar 7: Observabilidade Estruturada e o “Flywheel” Cognitivo



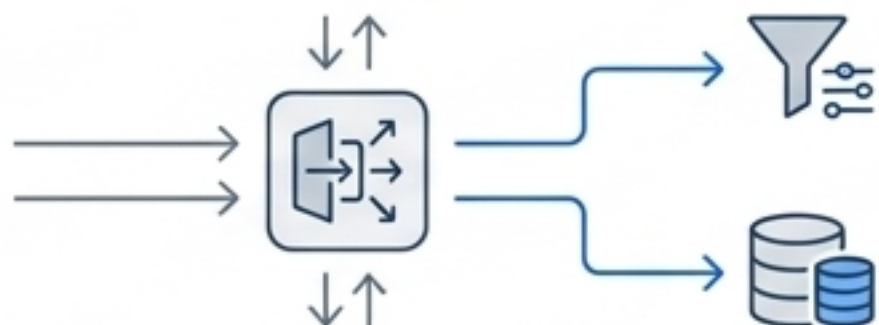
Validação em Larga Escala: A Operação AI-Native do PayPal

24.000 funcionários

Trilhões de transações mensais

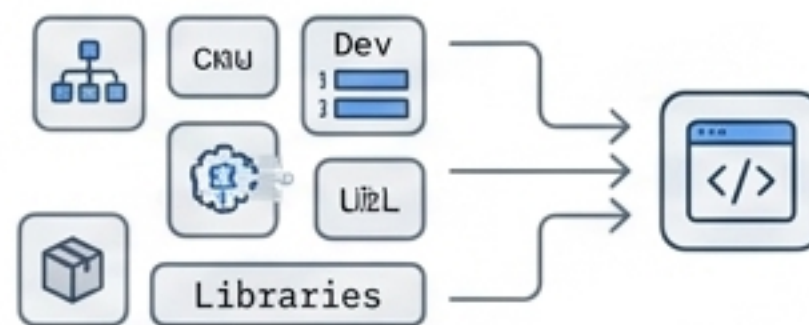
Gasto de \$2k/mês em tokens por Power User

Gateway Central (Vertex API)



Controle de tráfego corporativo, filtros de segurança e cache massivo de requests.

Marketplace MCP Interno



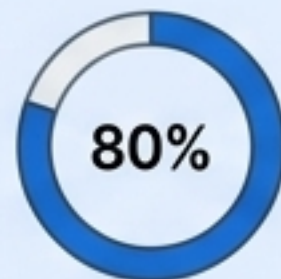
Desenvolvedores usam sub-agentes corporativos e bibliotecas restritas de inteligência.

Segurança de Supply Chain

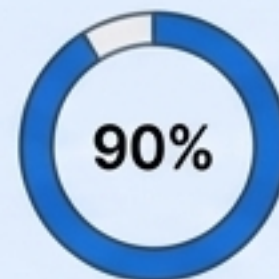


Cooldown de 7 dias para novos pacotes de registro (NPM/Python).

O Novo Dia a Dia



DEV TEMPO
COM IA



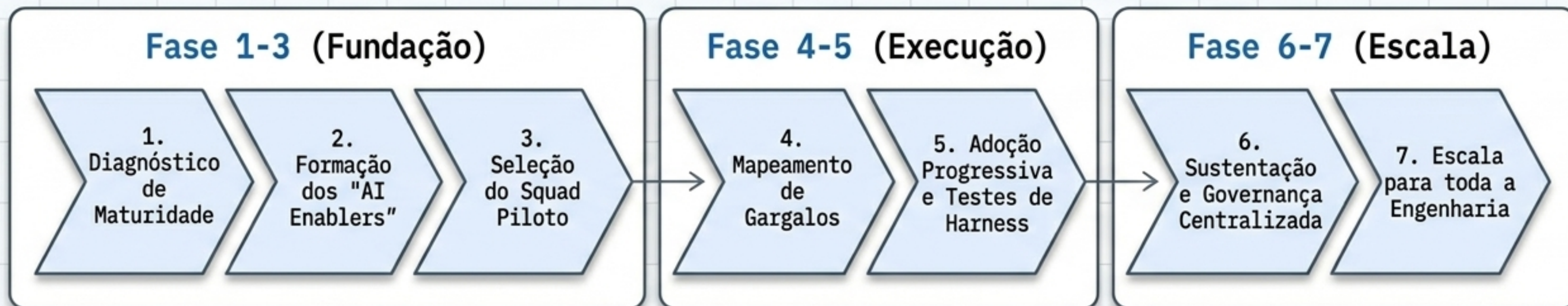
CÓDIGO
DUAL-REVIEW
(IA + HUMANO)

Devs passam 86% do tempo com IA, e 90% do código passa por dual-review (Revisão da IA + Revisão Humana).

Antipatterns Corporativos: Como NÃO Adotar IA

Antipattern	Sintoma	A Realidade/Solução
Teatro da IA	A diretoria compra licenças para todos e espera mágica sem treinamento.	Ferramentas abandonadas ou usadas como autocompletar glorificado.
Amplificador de Caos	IA inserida em repositórios sem testes unitários ou CI/CD sólidos.	Aumento reportado de +40% em bugs. A IA escala a má qualidade existente.
Shadow AI	Engenheiros usando LLMs não autorizados porque a infraestrutura da empresa é bloqueada demais.	Vazamento de dados sensíveis e quebra de compliance de segurança.

Framework de Adoção Corporativa: O Caminho Seguro



Empresas com adoção de sucesso levam em média **3 meses de calibração** antes de observar um salto sustentável de **~39% em produtividade**.

A Evolução do Dev: O Perfil "Product Engineer"

1. Definição (Humano)



Atua no Board (Linear/Slack), criando "Slices" de tarefas com forte contexto de produto e especificações.

2. Delegação (Cloud Agent)



O gatilho desperta um agente na nuvem. Ele faz o setup, processa as skills, implementa e cria o Pull Request isoladamente.

3. Auditoria (Humano)



O dev entra apenas na fase de Code Review, avaliando o PR gerado pelo agente junto a verificações de segurança automatizadas.

O desenvolvedor deixa de ser um digitador local para se tornar um orquestrador de sistemas complexos.

[STATUS: COMPILED]

Resumo da Execução: Engenharia de Harness

**Pare de comparar agentes de IA pelo modelo.
Comece a compará-los pela qualidade do Harness.**

```
const takeaways = [  
  'A consistência arquitetural do seu sistema sempre vencerá o talento  
  isolado do modelo.',  
  'Construa sua alavanca: Especifique antes de codar, verifique de forma  
  inteligente e proteja seus dados.',  
  'O futuro não pertence a quem digita mais rápido, mas a quem orchestra  
  agentes com mais segurança.'  
]
```